Paper Review

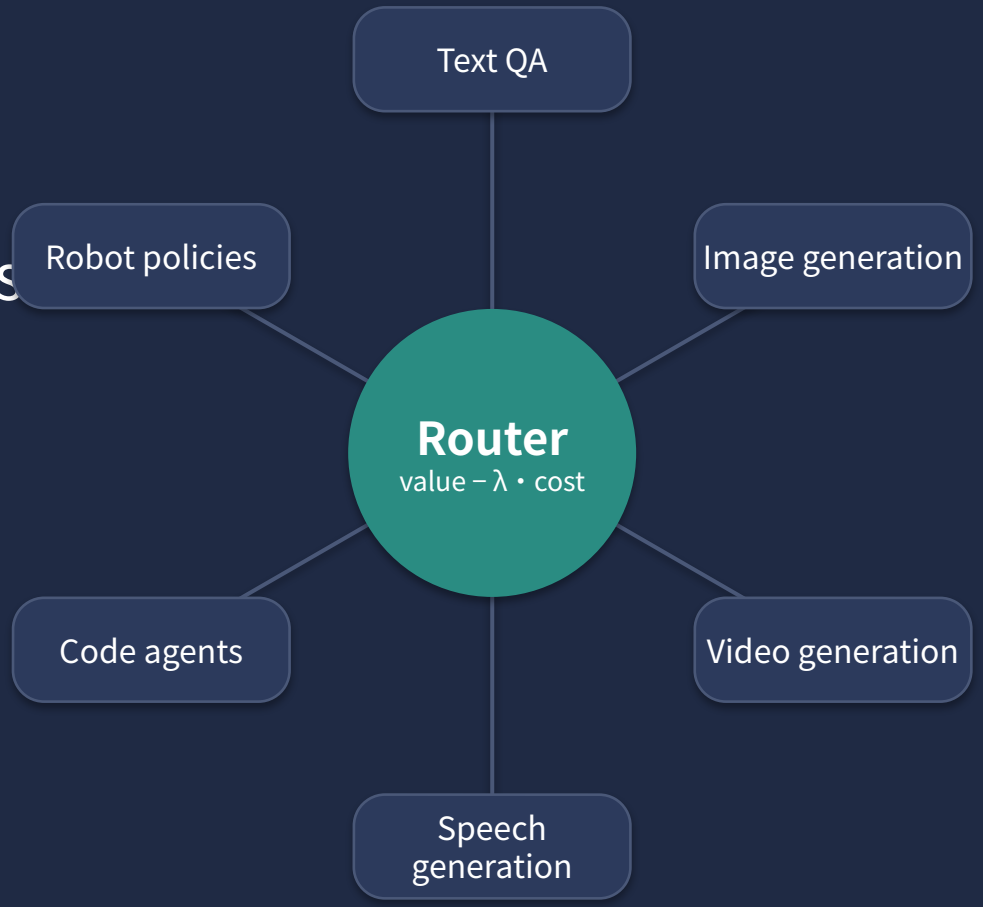
Route Anything

Model and Policy Routing in General-Purpos AI Systems

A Study of Model and Policy Routing for General-Purpose AI Systems

Pengfei Zhou, Yixing Ma, Hexin Wang ‡ , Weidong Tang,
Zhenglin Wan, Wangbo Zhao, Yang You ‡

NUS HPC-AI Lab · InfRec · Cardinal AI Lab
UC Berkeley · HKUST (‡ corresponding author)



arXiv 2611.05275 | First submitted 2026-11-03

In One Sentence

High ceiling, but the surest gain is “good-enough cheap routes”

The authors propose a unified theory of routing and run 452 candidate configurations across 25 benchmarks on **the same inputs** (374,401 scored records). Finding: the per-input selection ceiling is high, but learned routers capture only a small fraction of it in quality; **routing's most reliable benefit is a large efficiency gain at essentially unchanged quality.**

5.4 pts

Per-input oracle over best fixed candidate

24 benchmarks • 19,814 test inputs

+0.3 pts

Learned router over best fixed candidate

16 benchmarks; video gen +2.1, robotics +1.1

8.82×

Mean efficiency gain vs. reference model

Quality loss < 0.5 pts; code 25.5×, text QA 7.1×

Background & Problem

Each input needs a choice of model, generator or robot policy

Candidate pool: many options for the same input

Language models	Claude Opus 5.5 • GPT-6.1 Sol • DeepSeek V4.1 Flash • GPT-6 Astra
Image generation	FLUX 3 • Seedream 5.0 Pro • MAI-Image 2.6 • GPT Image 2.5 • Qwen-Image 3 Pro • Grok Imagine 2.0
Video generation	MiniMax H3 • Seedance 2.5 • Wan 3.0
Speech generation	Seed Audio 1.0 • Fish S2.1 Pro • Gemini 3.8 Flash TTS
Robot policies	π0.5 • GigaBrain • Spatial Forcing • X-VLA • X-WAM • PatchWAM • ME-Dex

Three questions the paper answers

Q1 — **How much can routing gain?**
Per-input oracle vs. best fixed candidate (§ 3)

Q2 — **How do learned routers perform?**
Cross-modal routers, robot scene routing, training objectives (§ 4)

Q3 — **When is a cheap route enough?**
Prices, reference models and certifiable quality floors (§ 5)

Prior routing is nearly LLM-only (cascades, preference, quality–cost); cross-modal efficacy is open.

Source: § 1 Introduction; Appendix B.1 (candidate list)

Unified Theory (§ 2)

Router value = best expected net-of-cost outcome given observables

Objective: $J\lambda(g) = E[Vg] - \lambda \cdot E[Dg]$ Efficiency: $\eta = E[V] / E[D]$ **Lemma 1:** efficiency gain $\Leftrightarrow Q + \eta B \cdot S > 0$, a condition independent of the quality floor $Q \geqslant -\varepsilon$

Theorem 2 • Optimal routing under a price

Each state picks only the candidate maximizing $v(s,u) - \lambda \cdot c(s,u)$; peak efficiency η^* is the unique root of $\Phi(\lambda)$ (Dinkelbach + LP duality).

Efficiency-optimal routing with floor $Q \geqslant -\varepsilon$ = routing at some price $\lambda\varepsilon$, set by the reference model and tolerance, not by hand-tuning or budgets.

Proposition 3 • Information ladder

$V^*(\text{benchmark}) \leqslant V^*(\text{type}) \leqslant V^*(\text{input}) \leqslant E[\max W]$ (per-input oracle)

Coarse states help only if they change the best candidate; the last gap is outcome noise that no observable state can predict.

Proposition 4 • Learn outcomes, not winners

Per-candidate regression of $E[V_u \mid x]$ (cross-entropy or squared error) attains $V^*(x)$.

Classifying "who wins" can lose nearly half the value range; awarding ties to faster candidates makes it worse.

Proposition 5 • Certifiable quality floor

On held-out samples, test $Q < -\varepsilon$ in a fixed order and stop at the first failure; if none passes, fall back to the reference model.

Guarantees $P(Q < -\varepsilon) \leqslant \alpha$ with no multiple-comparison correction.

Certifiable Quality Floor (Prop. 5)

Price-path tests, stopped at first failure, certify the quality floor

1 Build the price path
On fitting data, raise λ from 0 to η* to get g1…gK: fitted value and cost decrease, efficiency increases.

2 Held-out testing
Compute a p-value for each gk (Hoeffding / empirical-Bernstein) under the null Q < −ε.

3 Stop at first failure
Pick the last consecutively passing router; if g1 fails, return the reference g0.

✓ Guarantee
P(Q < −ε) ⩽ α; fixed-sequence testing needs no multiplicity correction and also yields a "certified price."

Figure: paper Figure 2 (figures/fig01.png); source: § 2, Appendix A.5

Data: All-Modality Trace Matrix (§ 3, Appendix B)

25 benchmarks, 452 candidate configs, all scored on the same inputs

25
benchmarks, 7 groups

452
candidate configs

374,401
scored runs

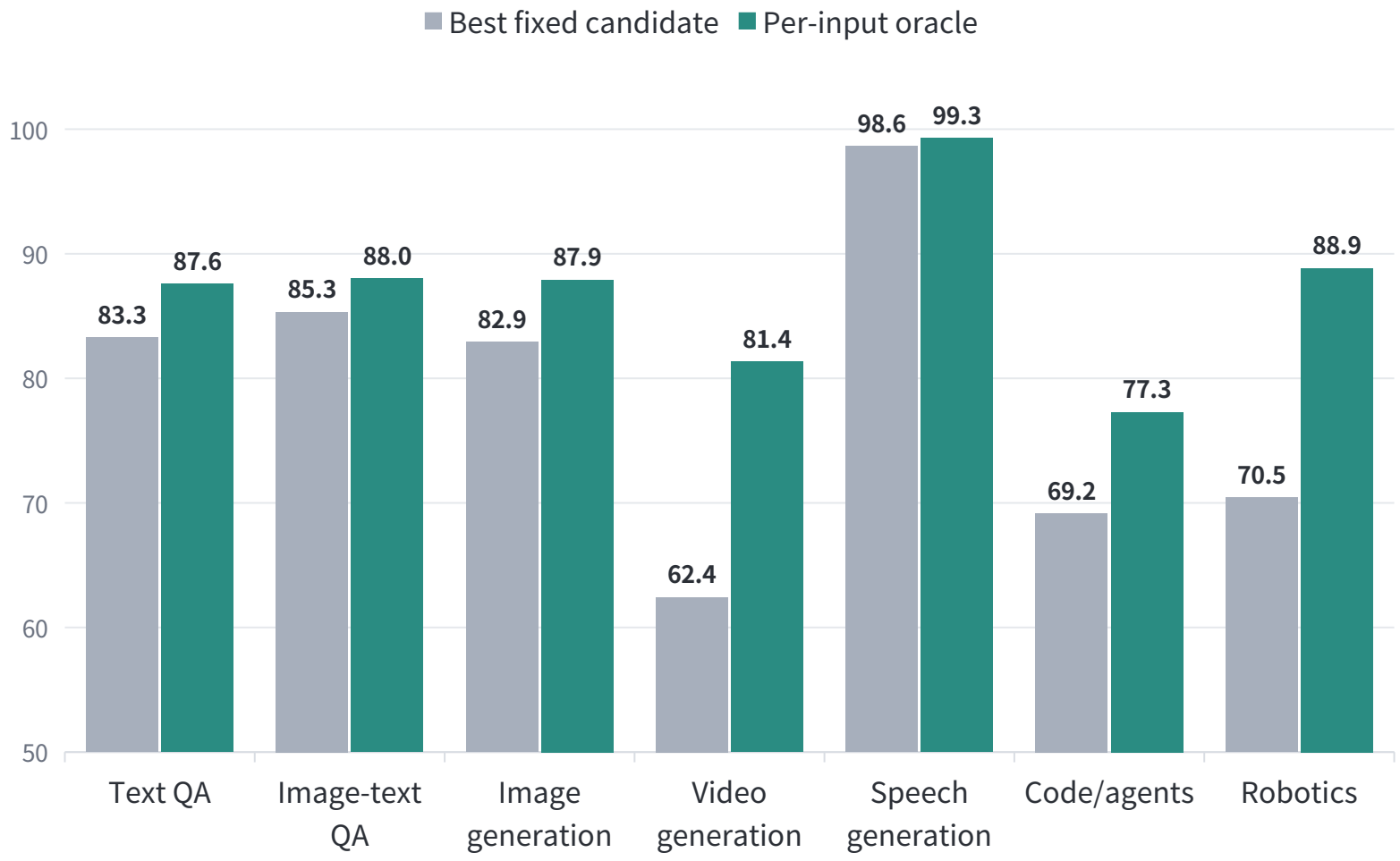
19,814
test inputs scored by all candidates

Group	Benchmarks	Candidates
Text QA	MMLU-Pro • MMLU-Redux • SuperGPQA • GPQA • AA-LCR • AA-Omniscience • HLE (text)	3–4
Image-text QA	MMMU-Pro • MMMU • VLM-ExecRouterBench	3
Image generation	GenEval2 • Qwen-Image-Bench • LongT2I • OneIG • WISE	4–6
Video generation	VBench-2.0	3
Speech generation	Seed-TTS (EN / ZH)	3
Code & office agents	AA-Briefcase • AutomationBench • GDP.pdf • GDPval • SciCode • Terminal-Bench	4
Robot manipulation	RoboTwin 2.0 (50 bimanual tasks, clean / randomized scenes)	7

Split 70/10/20 by source group; RoboTwin's 2,000 test scenes come from 10 tasks never seen in training. Prices: the matrix records only image/speech prices and robot inference time; code and text prices come from CodeRouterBench and RouterBench.

Finding 1 • Potential Gain (§ 3)

Per-input oracle beats best fixed by 5.4 pts; video/robots near 19



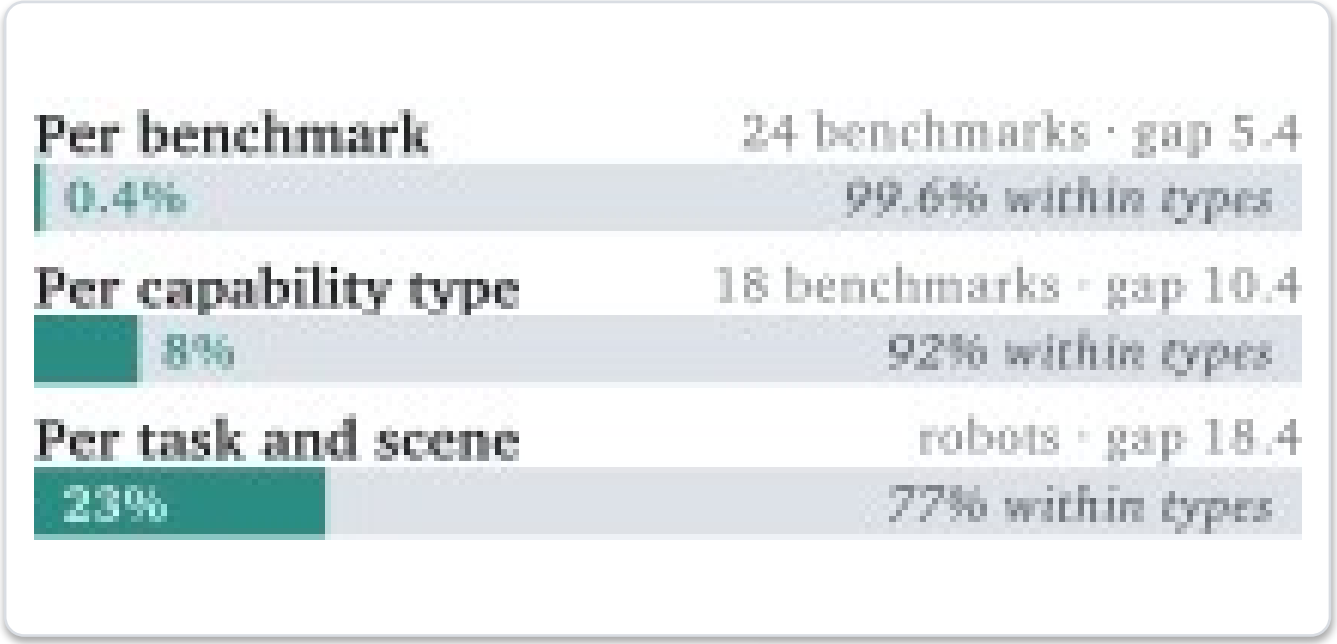
+5.38

Overall: 82.60 → 87.98

24 benchmarks • 19,814 inputs

Video generation	+18.93
Robot manipulation	+18.40
Code/agents	+8.11
Speech generation	+0.68

Task type alone captures little: most of the gap lies within types



Caveat: part of the gap is noise

AA-LCR runs each item 3 times; scoring candidates by the 3-run mean erases the oracle's 2.4-pt edge over the fixed candidate.

Post-hoc choice per benchmark

82.60 → 82.62, capturing only 0.4% of the gap

Post-hoc choice over 107 capability leaves

77.89 vs fixed 77.03 vs oracle 87.44 → 8%

Robotics: by task × scene condition

Captures 23% of the 18.4-pt gap

Code: CodeRouterBench public routers

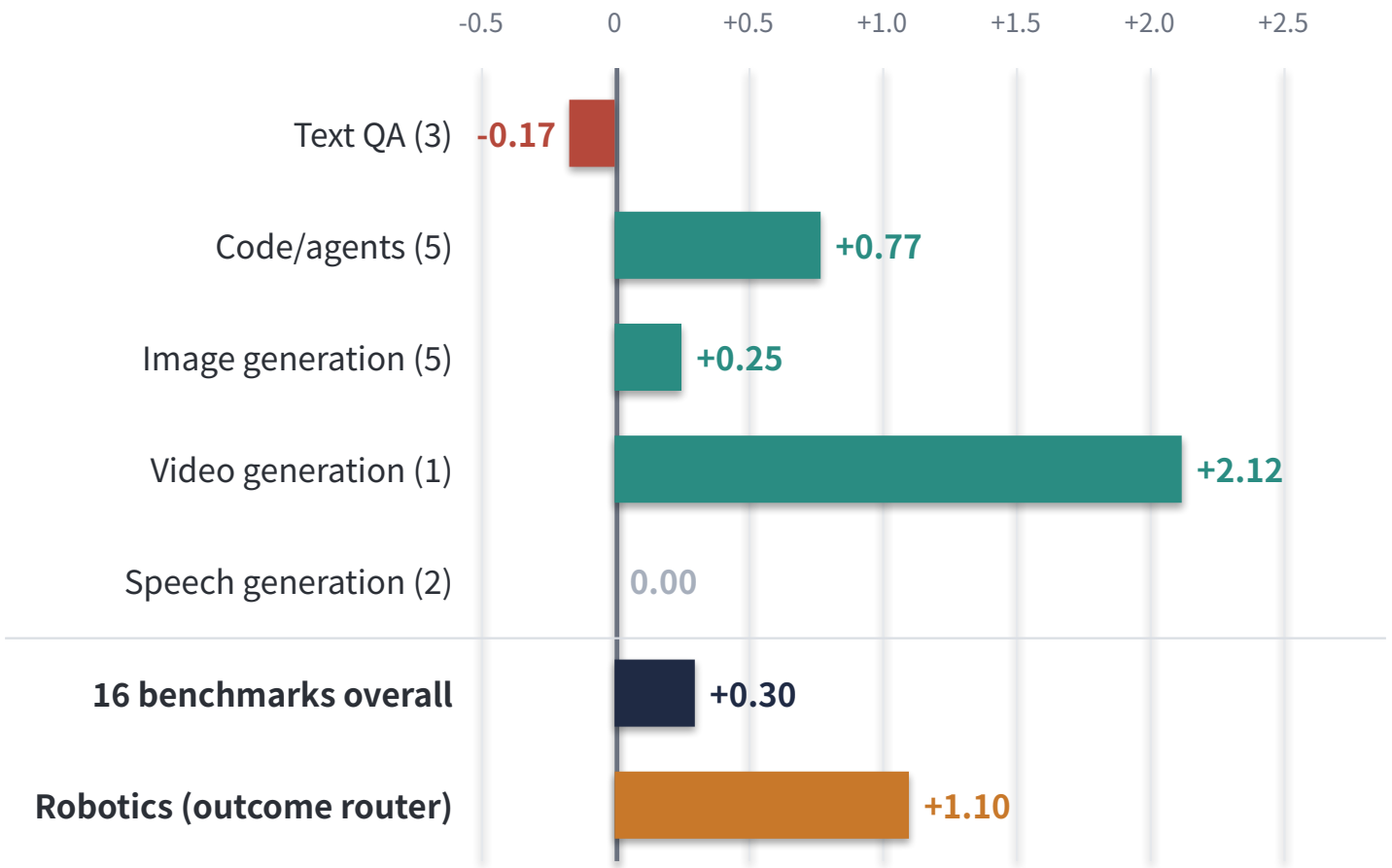
55% of perfect-information value with type unknown; only 6–8% within types

Figure: paper fig02.png (§ 3); data: § 3 Finding 1, Appendix C.5

Finding 2 • Learned Routers (§ 4.1)

Learned routers gain only 0.3 pts over best fixed, on a few benchmarks

Router − fixed candidate (pts, test set, single training seed)



79.98

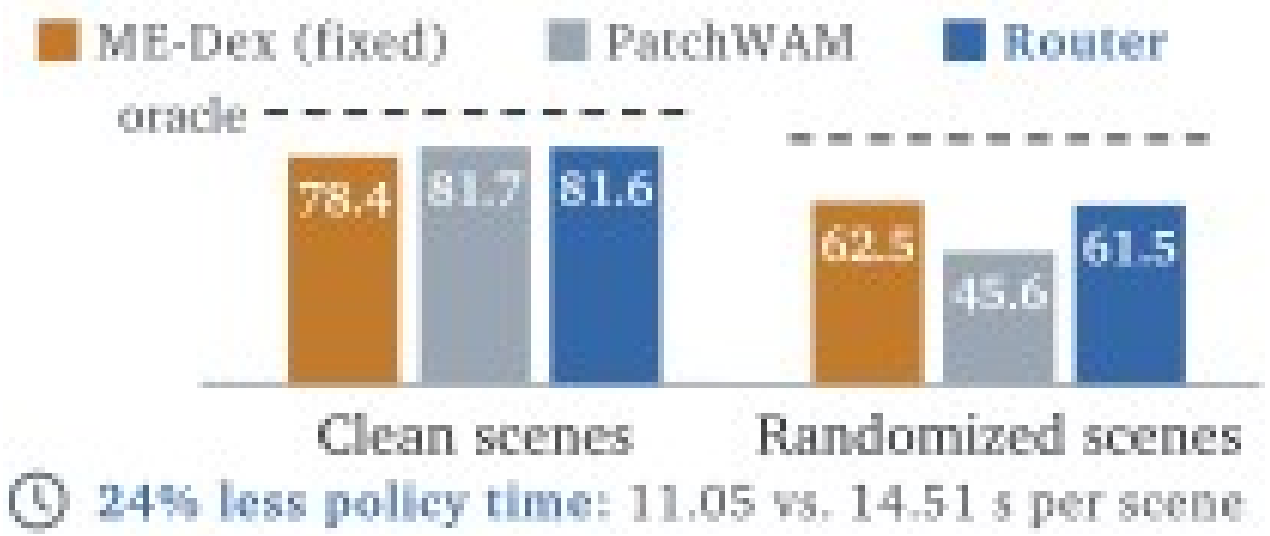
Capability-tree router vs fixed 79.68 / oracle 84.80

Captures only 6% of the oracle gain

- Leaf accuracy is 86–89%, yet 83–86% of inputs still go to the fixed candidate: within a benchmark, nearly every type has the same best candidate.
- Per benchmark: Terminal-Bench +8.3 (only 36 inputs), VBench-2.0 +2.1, OneIG +1.1; AA-LCR −2.4.
- Image generation: of 31 designs and 124 configs, none significantly beats the fixed generator; 44 are significantly worse.
- Untrained Qwen3.5-4B picking candidates itself: 1.15 pts below the fixed candidate.

Finding 3 • Robot Scene Routing (§ 4.2)

Camera router ties best per condition; −24% policy time



Input: instruction + three 240×320 camera frames + robot state; 7 sigmoid heads predict the success rates of 7 VLA policies; no task identity; test scenes come from 10 unseen tasks.

71.55% vs 70.45%

Overall success +1.10, 95% CI [−0.40, +2.55] (spans 0)

−24% policy time

11.05 s vs 14.51 s / scene → 1.33× success per policy-second

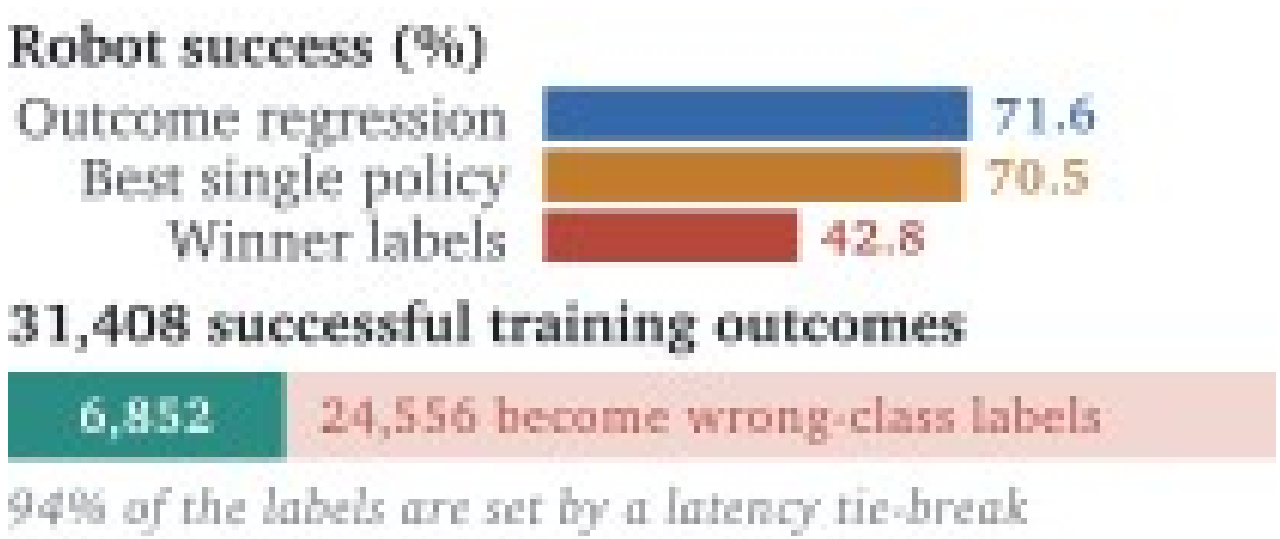
929 / 1000 • 842 / 1000

Clean scenes → PatchWAM; randomized scenes → ME-Dex

Figure: paper Figure 4 (figures/fig03.png); data: § 4.2, Tables 3, 5

Finding 4 • Training Objective (§ 4.3)

Learn outcomes, not “who wins” : winner labels drop success to 42.8%



Several policies succeed at once
Ties broken by inference latency

94% of labels decided by latency
24,556 / 31,408 successes become wrong-class targets

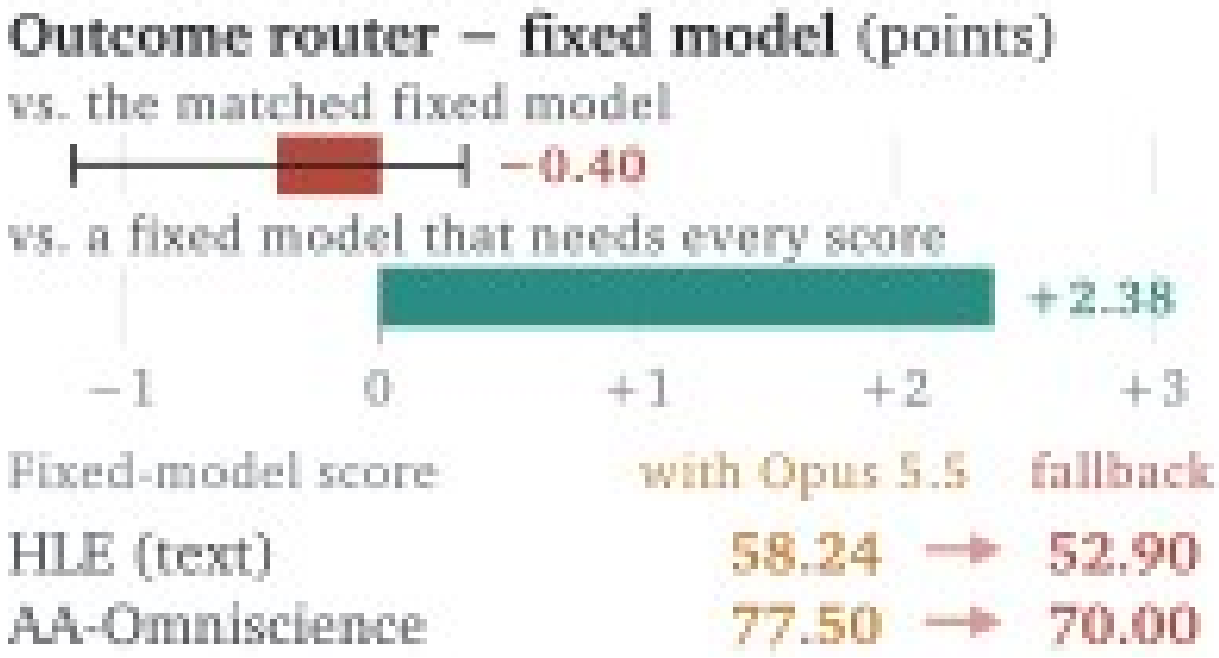
Classifier favors fast but often-failing policies
Mean 1.9 s / scene, 42.8% success

Prop. 4(b) counterexample
A slow policy with 0.9 success beats a fast 0.5-success policy on the label with probability only $0.9 \times 0.5 = 0.45$.

Across 16 benchmarks, the best of 12 generative winner-label schemes scores 78.87, still below the fixed candidate's 79.44.

Finding 5 • Matched Comparison (§ 4.4)

Poorly built baselines create spurious gains: −0.40 becomes +2.38



68.30 → 65.52

A baseline requiring "a score on every training input" drops Opus 5.5 (HLE lacks only 11 / 1,122 scores)

Fix

Select and evaluate the fixed candidate with the router's own fallback rule (GPT-6 Astra results when Opus 5.5 is missing); the spurious gain vanishes.

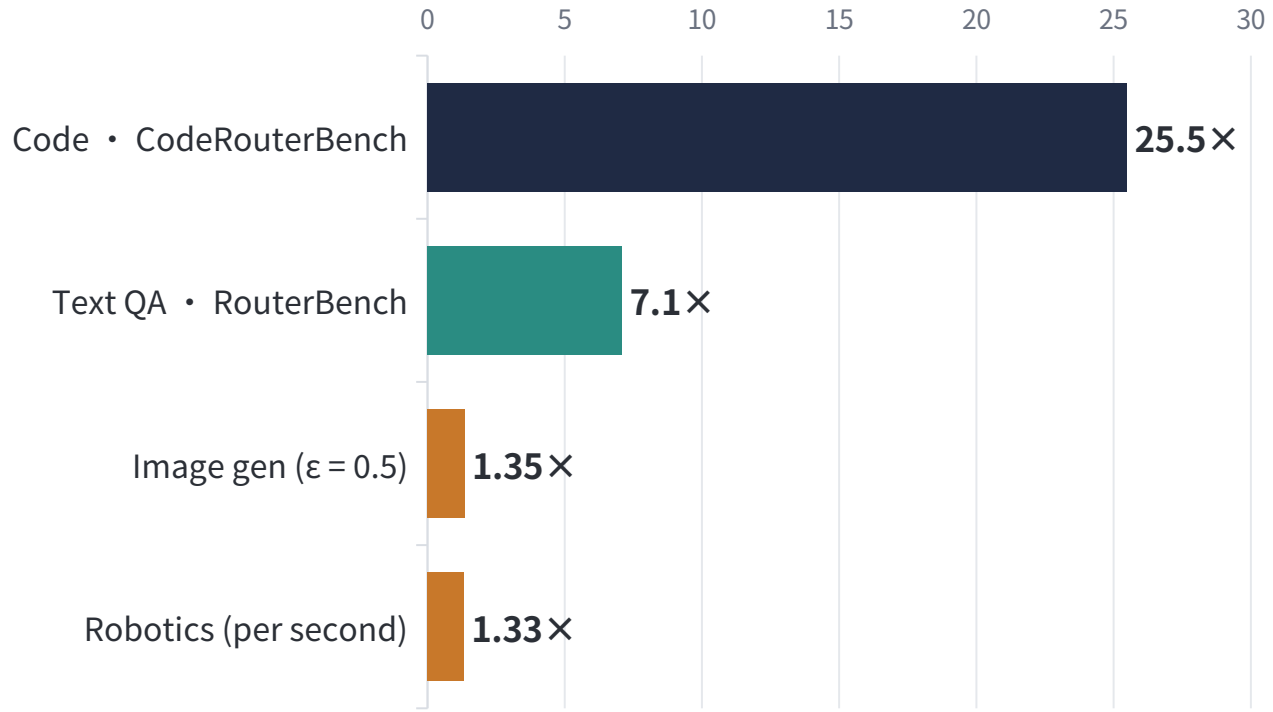
Likewise for portfolio selection: pairing two portfolios on the same tasks picks the best added model on CodeRouterBench 96.4–97.1% of the time; frozen averages reach only 86.3–94.3%.

Figure: paper fig05.png; data: Table 4, § 4.4, Appendix C.3

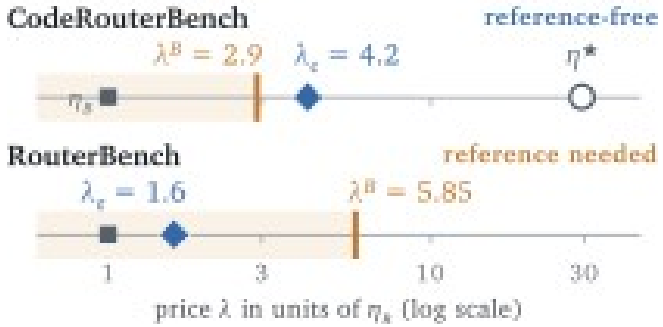
Finding 6 • Cheap Routes (§ 5.1)

Quality roughly unchanged; efficiency 8.82× the reference on average

η / η_B: efficiency ratio vs. reference model (test set)



Quality Δ: code −0.44 (CI ∋ 0), text −0.40, image −0.47, robot +1.10; 8.82× = arith. mean of 4 ratios; 25.5×, 7.1× from external CodeRouterBench/RouterBench.



Admission price gates reference calls (Cor. 6)

Code: λ^B = 2.9 η_B < λε = 4.2 η_B, so zero-loss routing never calls Opus. Text: λ^B = 5.85 > 1.6, so it must cascade back to GPT-4.

Fitted ≠ guaranteed

Fitted-only zero-loss floors hold on just 12.5% of splits; certified via Prop. 5, they hold on 200 / 200, median efficiency 19.4×.

Finding 7 • Spending More Within a Task (§ 5.2)

Retries, verifiers and finer signals must out-earn what they replace

Retries

0.49

Median ratio of the second model's success rate after a failure (θ = 1)

- All 42 ordered model pairs decline: failures leave harder tasks.
- Plan retries using post-failure conditional success rates (Prop. 8).

Verifiers

4.09× vs 0.30×

Free compiler checks vs list-priced LLM judges

- Qwen3-Max, θ = 0.5; compiler checks cost 2.4 pts of quality.
- 2.63× if the LLM judge is free; timed in seconds, Opus beats every verified cascade.

Finer states

⩽ 1 attempt

A signal saves at most the one attempt it replaces (Prop. 9)

- DeepSeek-v4-pro prediction costs 2.4 attempts: not worth it.
- A TF-IDF router costs only 0.06% of one attempt.

Assessment

Rigorous, candid; weak quality gains, and 8.82× needs unpacking

Strengths

■ Theory maps onto experiments: the information ladder explains "accurate classification, small gain"; Prop. 4 explains winner-label failure; Cor. 6 explains "zero loss without calling Opus."

■ Honest measurement: reports that none of 124 image-routing configs wins significantly; exposes spurious baseline gains; separates fitted from certified floors.

■ Brings VLA policy selection into a unified framework: scene-level dispatch from pixels saves time without lowering success rates.

■ Thm 2(ii) + Prop. 5 give deployable pricing and stopping rules; λ no longer needs hand-tuning.

Limitations / Caveats

▲ Weak quality gains: +0.3 over 16 benchmarks; the robot +1.10 has a 95% CI of [−0.40, +2.55].

▲ 8.82× is an arithmetic mean of four ratios; 25.5× / 7.1× come from external benchmarks, ~1.3× within the authors' own matrix.

▲ The oracle gap includes unpredictable noise (AA-LCR), so 5.4 pts is optimistic; some benchmarks are tiny (AA-Briefcase: 1, Terminal-Bench: 36).

▲ Theory assumes routing leaves the state distribution unchanged and omits step-wise routing for long-horizon agents; certification holds only relative to the stated reference (3.18× vs CellBest).

Conclusion (§ 7)

Three rules for routing practitioners

1

Compare candidates on the same inputs

Use the matched best fixed candidate as the baseline, so missing values and fallback rules cannot create spurious gains.

2

Train routers on per-candidate outcomes

Not on winner labels; look for gains within task types.

3

Fix the reference model and tolerance first, then rank routes

and certify the quality floor on held-out inputs.

In one line: cross-modal routing has a high ceiling (+5.4 pts), but learned routers capture only a small part so far (+0.3 pts); the most reliable gain is good-enough cheap routes (8.82×).